

# Laboratório 3 - Máquinas de estados finitos (FSM)

## Objetivos

1. Experimentar a descrição em VHDL de circuitos sequenciais;
2. Reforçar os conceitos de flip-flops e registradores;
3. Reforçar os conceitos e experimentar a descrição em VHDL de máquinas de estados finitos;
4. Pôr em prática conceitos aprendidos na disciplina Circuitos Digitais - Teoria.

## Introdução

Nesta aula iremos implementar e simular o projeto de circuitos sequenciais por meio de máquinas de estados finitos. Serão abordadas descrições em VHDL por portas lógicas e comportamentais.

## Circuitos Sequenciais x Circuitos Combinacionais

A saída de um circuito combinacional é uma função que depende apenas das entradas atuais do circuito. Um circuito combinacional não tem memória – não podemos armazenar bits em um circuito combinacional e depois ler esses bits. Os circuitos combinacionais em si são muito limitados em sua utilidade. Por outro lado, usualmente os projetistas usam os circuitos combinacionais como parte de circuitos maiores, chamados circuitos sequenciais – circuitos que realmente têm memória. Um circuito sequencial é um circuito cujas saídas dependem não somente das entradas atuais, mas também do seu estado atual, que é o conjunto de todos os bits armazenados no circuito. O estado do circuito, por sua vez, depende da sequência passada de valores que apareceram nas entradas do circuito.

O VHDL é uma linguagem não-sequencial, portanto, faz-se necessário uma estrutura que o torne sequencial, para auxiliar na implementação de circuitos sequenciais. A estrutura que permite isto é o PROCESS, cuja sintaxe pode ser vista no Quadro 1.

```
PROCESS (A, B); --entre parenteses estão os sinais da lista de sensibilidade
BEGIN
    --comandos
END PROCESS;
```

**Quadro 1** – Sintaxe da estrutura PROCESS da linguagem VHDL.

Conceito importante: O trecho entre BEGIN e END é executado sequencialmente (a ordem em que os códigos estão apresentados importa). Por sua vez, o PROCESS é executado concorrentemente com as demais declarações. O PROCESS é invocado quando há uma mudança em algum sinal de sua lista de sensibilidade.

Uma vez definida a estrutura que executará um código sequencial em VHDL, precisamos definir quais comandos executados dentro desta estrutura que serão utilizados na implementação de um circuito sequencial. Uma forma de fazer isso é com o uso de outra estrutura do VHDL: a estrutura IF-ELSE-ELSIF, a qual é executada de forma sequencial, podendo somente ser utilizada em regiões de código sequencial. Funciona de forma semelhante à estrutura padrão de IF de linguagens de alto nível, obedecendo a sintaxe ilustrada no Quadro 2.

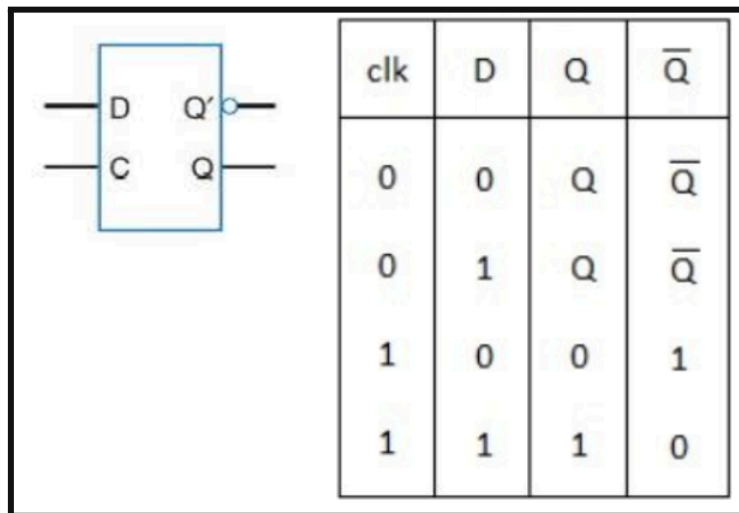
```
IF (condicao1) THEN
    --Comandos caso essa condição seja verdadeira
ELSIF (condicao2) THEN
    --Comandos caso essa condição seja verdadeira
ELSE
    --Comandos caso nenhuma condição anterior seja verdadeira
END IF;
```

**Quadro 2** – Sintaxe da estrutura IF-ELSE-ELSIF da linguagem VHDL.

## Latches

Um latch é um dispositivo de armazenamento temporário que tem dois estados estáveis. Os latches são similares aos flip-flops por serem dispositivos biestáveis, ou seja, podem permanecer em um de dois estados estáveis usando uma configuração de realimentação, na qual as saídas são ligadas nas entradas opostas. A principal diferença entre os latches e os flip-flops é o método usado para a mudança de estado.

**Latch D (sensível ao nível)** - O latch D, sensível ao nível, ilustrado na Figura 1, é um dispositivo que funciona da seguinte forma. Se a entrada C do latch D for igual a '0', a saída Q do latch D não será modificada, ou seja, seu estado interno é armazenado. Por outro lado, se a entrada C do latch D for igual a '1', a saída Q do latch será igual ao valor da entrada D: o latch é setado se D='1' e ressetado se D='0'. O Quadro 3 apresenta a descrição compartamental de um latch D e a Figura 2 apresenta um diagrama de tempo obtido a partir da simulação do código VHDL do Quadro 3.



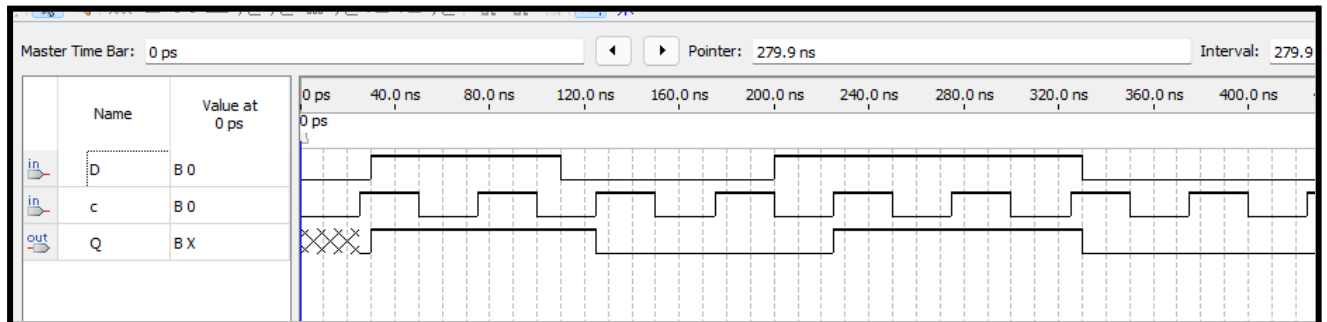
**Figura 1:** Representação em bloco e tabela verdade do latch D.

```

LIBRARY ieee;
USE ieee.std_logic_1164.all ;
ENTITY latchD IS
PORT(D, c : IN BIT;
      Q : OUT BIT);
END;
ARCHITECTURE behav OF latchD IS
BEGIN
PROCESS (D, c)
BEGIN
  IF (c = '1' AND D = '1') THEN
    Q<='1';
  ELSIF (c = '1' AND D = '0') THEN
    Q<='0';
  END IF;
END PROCESS ;
END;

```

**Quadro 3:** Descrição de um latch D (sensível ao nível).

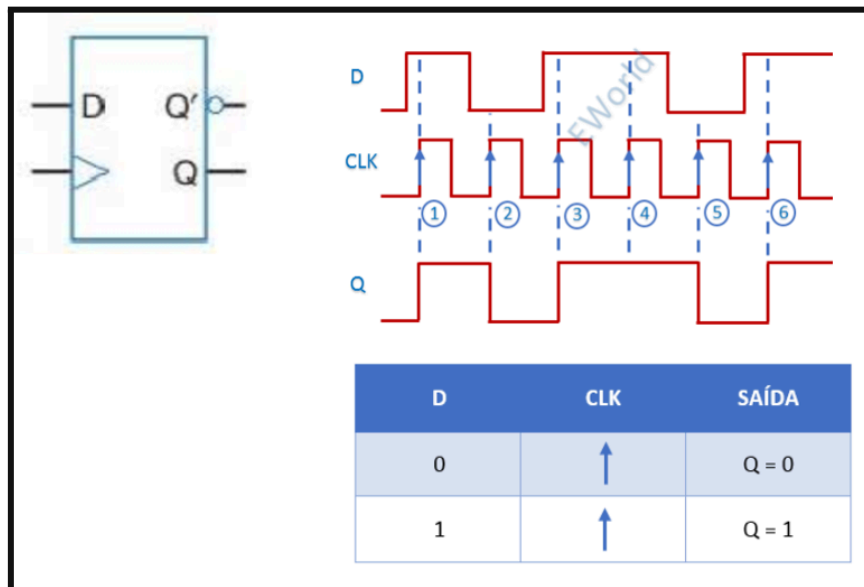


**Figura 2:** Formas de onda obtidas pela simulação da descrição VHDL do latchD.

## Flip-flops

Um problema dos latches é que se o valor da entrada D mudar enquanto o sinal C está ativado, ele ainda é levado em consideração. Para evitarmos este problema, utilizamos os dispositivos flip-flops, os quais só registram o bit quando há mudanças de nível em sua entrada C (também chamada de *clock*, ou CLK).

**Flip-flop D** - O flip-flop D é um dispositivo sensível à mudança de nível de sua entrada C (ou CLK). No caso em que o flip-flop D é sensível a uma borda de subida (mudança de CLK='0' para CLK='1'), a sua saída Q será igual a entrada D sempre que houver uma borda de subida em CLK. Em todos os outros casos, a saída Q não sofrerá alterações, ela apenas armazenará seu estado anterior. A Figura 3 ilustra a representação em blocos do flip-flop D; suas formas de onda, contendo as entradas D e CLK e a saída Q; e sua tabela verdade. O Quadro 4 apresenta a descrição VHDL de um flip-flop D e a Figura 4 ilustra as formas de onda obtidas pela simulação da descrição VHDL do flip-flop D.



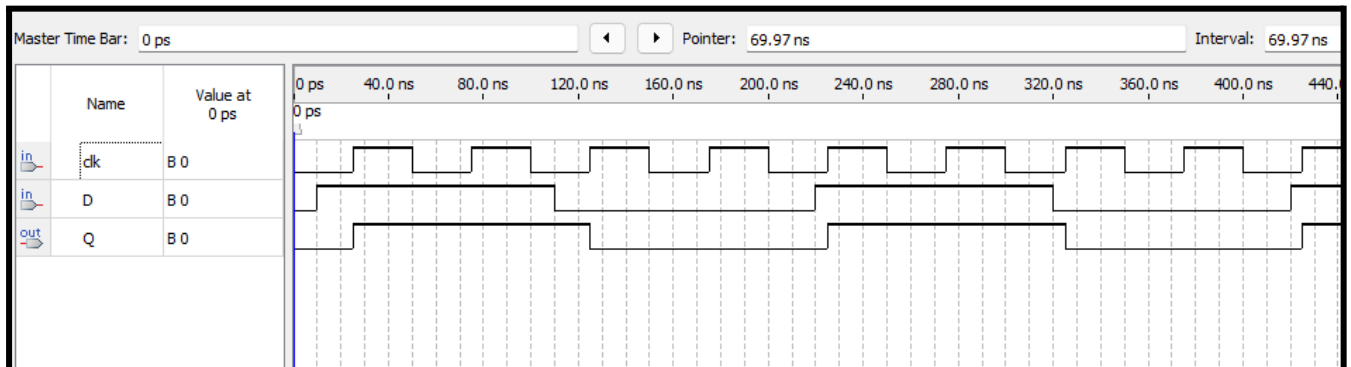
**Figura 3:** Representação em bloco, formas de onda e tabela verdade do flip-flop D.

```

LIBRARY ieee;
USE ieee.std_logic_1164.all ;
ENTITY flipflopD IS
PORT(D, clk : IN BIT;
      Q : OUT BIT);
END;
ARCHITECTURE behav OF flipflopD IS
BEGIN
PROCESS (D, clk)
BEGIN
  IF (clk ' EVENT AND clk = '1' AND D = '1') THEN
    Q<='1';
  ELSIF (clk ' EVENT AND clk = '1' AND D = '0') THEN
    Q<='0';
  END IF;
END PROCESS ;
END;

```

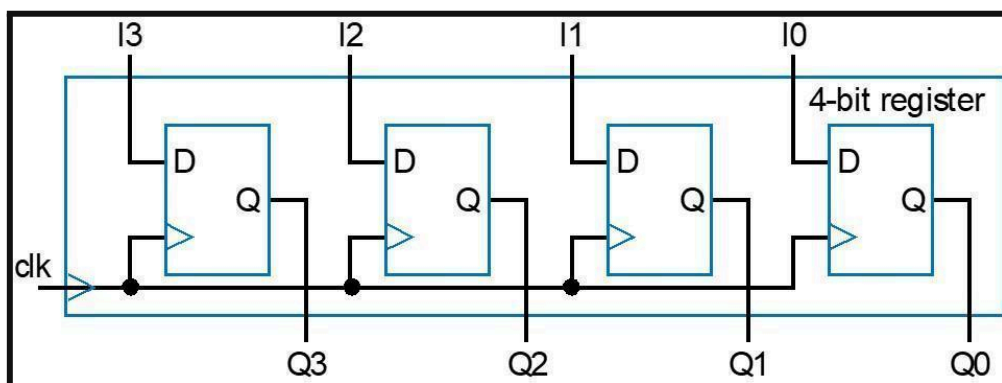
**Quadro 4:** Descrição de um flip-flop D.



**Figura 4:** Formas de onda obtidas pela simulação da descrição VHDL do flip-flop D.

## Registradores

Registradores são componentes sequenciais usados para armazenar múltiplos bits. Um registrador básico é construído empregando uma sequência de latches ou flip-flops. Por possuírem um estado em que a saída é indesejável (estado em que  $SR = 11$ ), latches e flip-flops SR não são usados para criar registradores. Em geral, são usados latches ou flip-flops D. Na Figura 5, podemos observar um registrador construído com flip-flops D, utilizado para armazenar 4 bits. Sempre que houver uma borda de subida na entrada CLK, todos os 4 flip-flops irão carregar, simultaneamente, suas entradas I0, I1, I2 e I3.



**Figura 5:** Registrador de 4 bits construído a partir de flip-flops D.

## Máquina de estados finitos

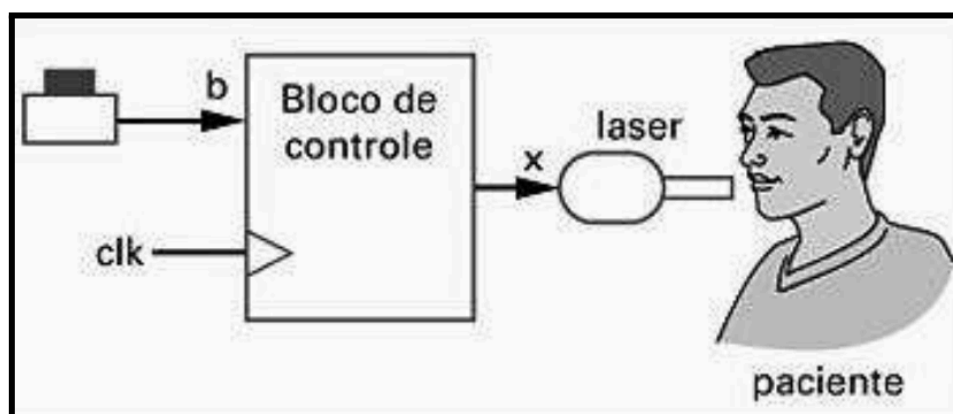
Uma máquina de estados finitos, ou FSM (do inglês, *finite state machine*), é um formalismo matemático que consiste em:

- Um conjunto de estados.
- Um conjunto de entradas e um conjunto de saídas.
- Um estado inicial, isto é, um estado para se começar quando o sistema é energizado. O estado inicial de um sistema pode ser indicado por meio de uma linha com seta, chamada de aresta, que não tem estado de origem e aponta para o estado inicial. Uma FSM pode ter apenas um estado inicial.
- Uma descrição que indique para qual estado devemos ir a seguir, com base no estado atual e nos valores das entradas. Por convenção, utiliza-se arestas orientadas ou dirigidas juntamente com as condições de entrada associadas, as quais nos dizem qual é o próximo estado. Essas arestas são conhecidas como *transições*.
- Uma descrição de quais são os valores de saída que devem ser gerados em cada estado. Em uma FSM, a atribuição de uma saída é conhecida como *ação*.

Usamos uma representação gráfica, conhecida como *diagrama de estados*, para representar uma FSM. Os diagramas de estados são bastante utilizados por permitirem uma visualização ampla do comportamento de uma máquina de estados finitos.

O projeto de um sistema digital representado por uma FSM gera um circuito digital conhecido como *bloco de controle* ou *controlador*. Este tipo de processo se aplica quando desejamos projetar um circuito digital que possui comportamento sequencial.

Vamos utilizar uma FSM para representar o projeto de um sistema digital que deve ser usado como parte de um sistema de cirurgia laser. Tal sistema funciona acionando um laser durante um intervalo de tempo preciso. Uma arquitetura genérica para este sistema está mostrada na Figura 6.

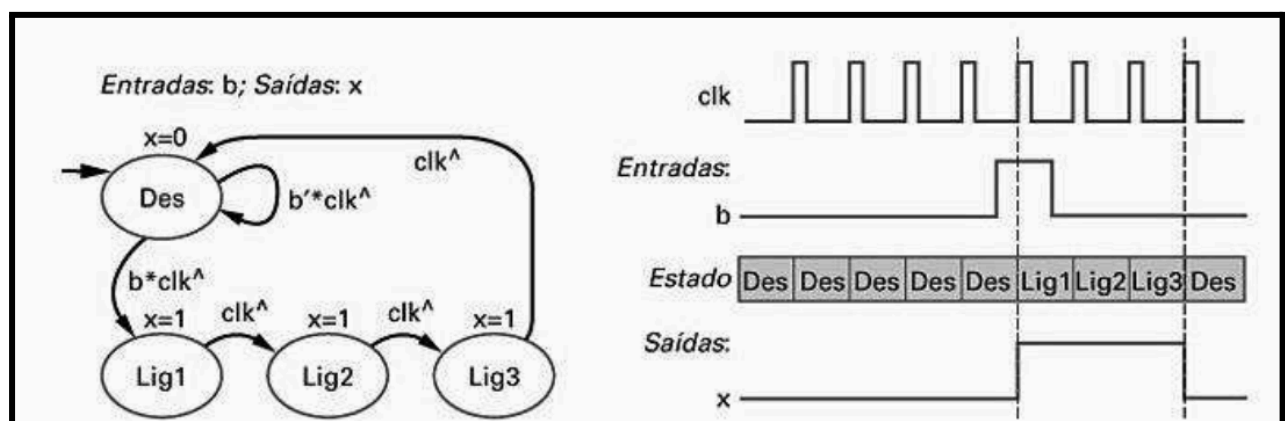


**Figura 6** - Sistema temporizador de laser.

Um cirurgião ativa o laser pressionando um botão. Assuma, então, que o laser deve permanecer ativado por exatamente 30 ns. Assumindo que o período do nosso relógio é 10 ns, então 30 ns significa 3 ciclos de relógio (clock). Precisamos projetar um componente de

bloco de controle que, tendo detectado que  $b=1$ , mantém  $x$  em nível alto por exatamente 3 ciclos de relógio, ativando o laser por exatamente 30 ns.

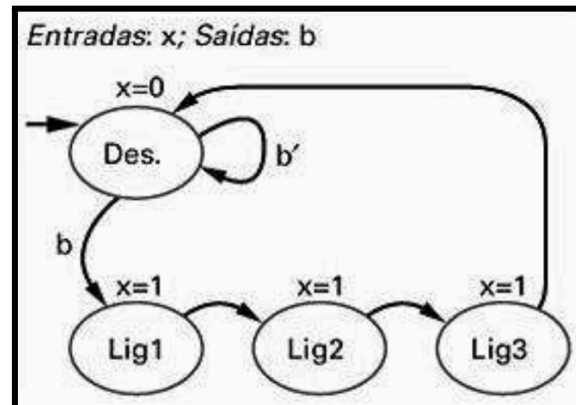
Para projetar este sistema, vamos criar uma FSM com quatro estados: *Desligado* (*Des*), *Ligado1* (*Lig1*), *Ligado2* (*Lig2*) e *Ligado3* (*Lig3*). A saída  $x$  deverá ser 0 durante um ciclo, no estado *Des*, e 1 durante três ciclos, nos estados *Lig1*, *Lig2* e *Lig3*. Iremos impor condições de entradas às transições. Para que ocorra uma transição do estado *Des* para o estado *Lig1*, devemos ter uma borda de subida no relógio e  $b=1$ . Acrescentaremos também uma transição do estado *Des* retornando ao próprio estado *Des*, com a condição de que ocorra uma borda de subida e  $b=0$ . As transições dos estados *Lig1* para *Lig2*, *Lig2* para *Lig3* e *Lig3* para *Des* ocorrem sempre nas próximas bordas de subida do relógio. O diagrama de tempo da Figura 7 mostra o comportamento de estado e de saída do sistema para os dados valores de  $b$ .



**Figura 7** - Sistema para três ciclos em nível alto: diagrama de estados (à esquerda) e diagrama de tempo (à direita).

A FSM deve ser interpretada como segue. Começamos com nosso estado inicial de *Desligado*. Permaneceremos nesse estado até que uma das duas transições que saem dele esteja na condição de verdadeira. Uma dessas transições tem por condição  $b'$  AND uma borda de subida de relógio ( $b' \cdot \text{clk}^\wedge$ ); nesse caso, a transição irá se dar retornando ao estado *Desligado*. A outra transição tem por condição  $b$  AND uma borda de subida de relógio ( $b \cdot \text{clk}^\wedge$ ); nesse caso, a transição ocorrerá indo para o estado *Lig1*. Permaneceremos no estado *Lig1* até que se torne verdadeira a condição necessária à transição de saída do estado, ou seja, uma borda de subida do relógio - nesse caso, a transição irá se dar passando-se ao estado *Lig2*. De modo semelhante, permaneceremos no estado *Lig2* até a próxima borda de subida do relógio, quando ocorrerá uma transição para o estado *Lig3*. Permaneceremos em *Lig3* até a próxima borda de subida do relógio, quando será feita uma transição de volta ao estado *Desligado*. No estado *Desligado*, temos a ação associada a saída  $x=0$ , ao passo que, nos estados *Lig1*, *Lig2*, e *Lig3*, temos a ação associada à saída  $x=1$ .

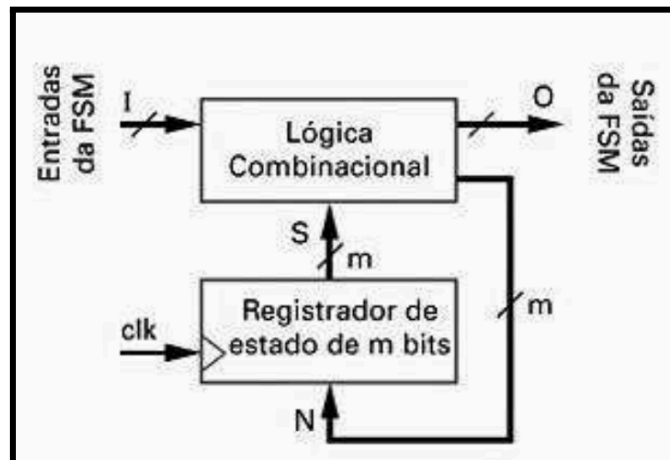
Desse modo, usando uma FSM, descrevemos precisamente o comportamento sequenciado no tempo que é esperado do sistema temporizador de laser. Podemos simplificar a notação da FSM, tornando implícita a borda de subida do relógio, como ilustrado na Figura 8.



**Figura 8** - Diagrama de estados do temporizador laser, no qual se assume que qualquer transição está sendo submetida a uma operação AND juntamente com a borda de subida do relógio.

## Arquitetura padrão do bloco de controle para implementar uma FSM na forma de circuito sequencial

A arquitetura de um bloco de controle padrão para uma FSM consiste em um registrador de estado e uma lógica combinacional, conforme ilustrado na Figura 9. O registrador de estado consiste em um registrador que contém um número binário que representa o estado atual. As entradas da lógica combinacional são as entradas da FSM e também as saídas do registrador de estado. As saídas da lógica combinacional são as saídas da FSM e também os bits do próximo estado que serão carregados no registrador de estado. Os detalhes da lógica combinacional determinam o comportamento do circuito. Na Figura 9, assumimos que o registrador de estado possui  $m$  bits de largura. A máquina de estados finitos, representada pela arquitetura da Figura 9, é conhecida como máquina de Moore.



**Figura 9** - Arquitetura de um bloco de controle padrão - visão geral.

## Projeto de bloco de controle

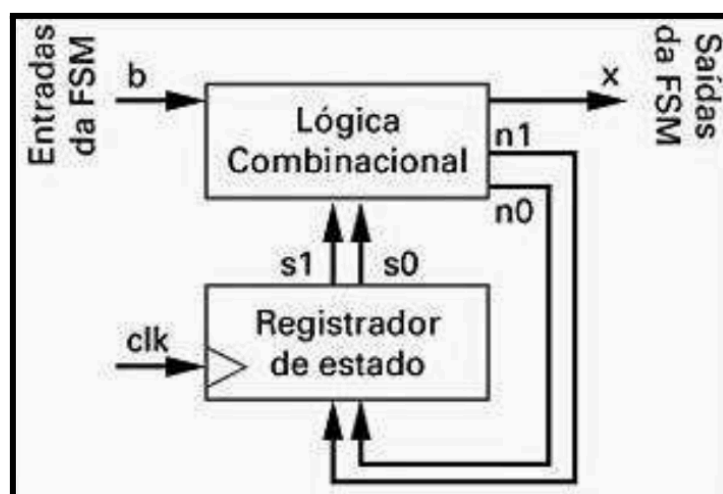
Podemos projetar um bloco de controle usando um processo de cinco passos, descrito na Figura 10. Iremos ilustrar esse processo usando como exemplo o sistema temporizador de laser.

| Passo   | Descrição                         |
|---------|-----------------------------------|
| Passo 1 | Capture a FSM                     |
| Passo 2 | Crie a arquitetura                |
| Passo 3 | Codifique os estados              |
| Passo 4 | Crie a tabela de estados          |
| Passo 5 | Implemente a lógica combinacional |

**Figura 10** - Processo de projeto de um bloco de controle.

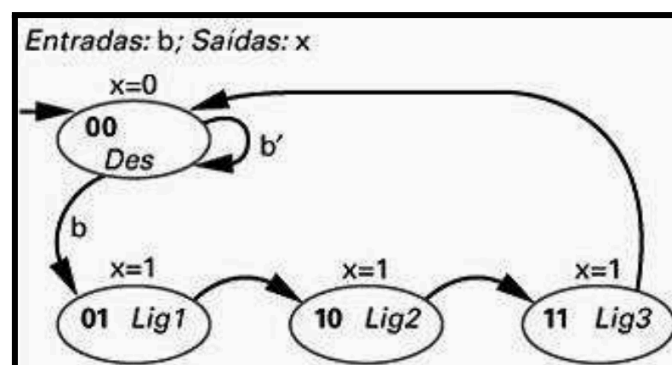
**Passo 1 - Capture a FSM.** A FSM já foi capturada e seu comportamento está ilustrado na Figura 8.

**Passo 2 - Crie a arquitetura.** A arquitetura do bloco de controle padrão para a FSM do temporizador de laser está ilustrada na Figura 11. O registrador de estado tem dois bits de largura para representar cada um dos quatro estados. A lógica combinacional tem a entrada externa  $b$  e as entradas  $s1$  e  $s0$  que vêm do registrador de estado, e tem a saída externa  $x$  e as saídas  $n1$  e  $n0$  que se dirigem ao registrador de estado.



**Figura 11** - Arquitetura de bloco de controle padrão para o temporizador laser.

**Passo 3 - Codifique os estados.** Podemos codificar os estados como segue. Des: 00; Lig1: 01; Lig2: 10 e Lig3: 11. Lembre-se, qualquer codificação não repetida é aceitável. O diagrama de estados com as codificações está mostrado na Figura 12.



**Figura 12** - Diagrama de estados do temporizador de laser com os estados codificados.

**Passo 4 - Crie a tabela de estados.** Dadas a arquitetura da implementação e a codificação binária de cada estado, podemos criar a tabela de estados para a lógica combinacional, como mostrado na Figura 13. Nas colunas de entrada, se listarmos primeiro

as entradas do registrador de estado, então poderemos ver facilmente quais linhas correspondem a quais estados. Na esquerda, preenchemos todas as combinações de entradas como se faz com uma tabela-verdade. Em cada linha, olhamos no diagrama de estados na Figura 8 para determinar as saídas apropriadas. Para as duas linhas que começam com  $s_1s_0=00$  (estado *Desligado*),  $x$  deve ser 0. Se  $b=0$ , o bloco de controle deverá permanecer no estado *Desligado*, de modo que  $n_1n_0$  deverá ser 00. Se  $b=1$ , o bloco de controle deverá ir para o estado *Lig1*, de modo que  $n_1n_0$  deverá ser 01.

De modo semelhante, para as duas linhas que começam com  $s_1s_0=01$  (estado *Lig1*),  $x$  deve ser 1 e o próximo estado deve ser *Lig2* (independente do valor de  $b$ ) de modo que  $n_1n_0$  deverá ser 10. Completamos as últimas quatro linhas de modo semelhante.

| Entradas    |       |       |     | Saídas |       |       |
|-------------|-------|-------|-----|--------|-------|-------|
|             | $s_1$ | $s_0$ | $b$ | $x$    | $n_1$ | $n_0$ |
| <i>Des</i>  | 0     | 0     | 0   | 0      | 0     | 0     |
|             | 0     | 0     | 1   | 0      | 0     | 1     |
| <i>Lig1</i> | 0     | 1     | 0   | 1      | 1     | 0     |
|             | 0     | 1     | 1   | 1      | 1     | 0     |
| <i>Lig2</i> | 1     | 0     | 0   | 1      | 1     | 1     |
|             | 1     | 0     | 1   | 1      | 1     | 1     |
| <i>Lig3</i> | 1     | 1     | 0   | 1      | 0     | 0     |
|             | 1     | 1     | 1   | 1      | 0     | 0     |

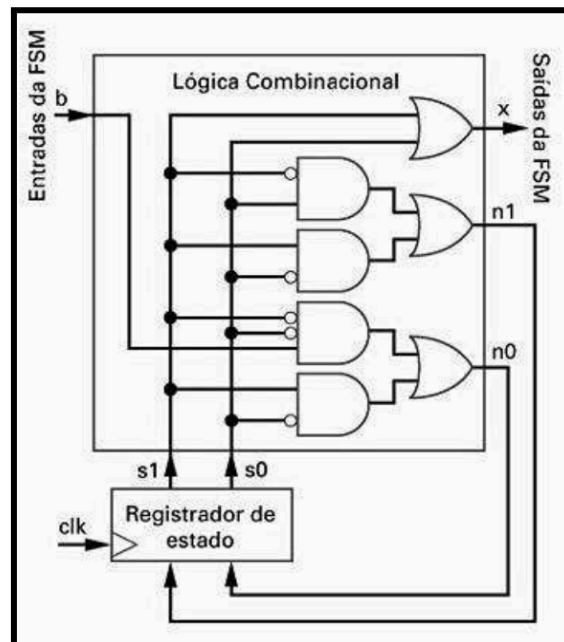
**Figura 13** - Tabela de estados para o bloco de controle do temporizador de laser.

Tenha cuidado e observe a diferença entre as entradas e saídas da FSM da Figura 8 e as entradas e saídas da lógica combinacional da Figura 14; esta última inclui os bits que se originam no registrador de estado e os que se destinam a este mesmo registrador.

**Passo 5 - Implemente a lógica combinacional.** Podemos terminar o projeto usando o processo de projeto de lógica combinacional, visto na disciplina de teoria de Circuitos Digitais. Da tabela verdade, obtemos as seguintes equações para as três saídas de lógica combinacional, já simplificadas.

$$\begin{aligned}
 x &= s_1 + s_0 \\
 n_1 &= s_1' s_0 + s_1 s_0' \\
 n_0 &= s_1' s_0' b + s_1 s_0'
 \end{aligned}$$

Obtemos, então, o circuito sequencial da Figura 9, que implementa a FSM.



**Figura 14** - Implementação final do bloco de controle para o temporizador de laser de três ciclos em nível alto.

## Descrição por portas lógicas em VHDL de uma FSM

No Quadro 5, pode-se observar o código com a descrição por portas lógicas em VHDL do bloco de controle para o temporizador de laser de três ciclos em nível alto, ilustrado na Figura 14. O Quadro 6 apresenta os componentes que contém o registrador de estado e o circuito combinacional do bloco de controle do Quadro 5.

A Figura 15 ilustra a forma de onda obtida com a simulação do código presente no Quadro 5. Note que a partir do instante em que a entrada *b* assume valor igual a 1, a partir da próxima borda de subida do relógio, a saída *x* permanece ativada por, exatamente, três ciclos de relógio.

```
library ieee;
use ieee.std_logic_1164.all;

entity LaserT is
port(b, clk : in bit;
      x      : out bit);
end LaserT;

architecture behavior of LaserT is
    signal n1, n0: bit; -- Proximo estado
```

```

signal s1, s0: bit; -- Estado atual
component reg2 is
  port(c, i1, i0 : in bit;
        q1, q0 : out bit);
end component;
component comb_circ is
  port(buttom, c, current1, current0 : in bit;
        next1, next0, laser_out : out bit);
end component;
begin
  u1 : reg2 port map(c => clk, i1 => n1, i0 => n0, q1 => s1, q0 => s0);
  u2 : comb_circ port map(buttom => b, c => clk,
                        current1 => s1, current0 => s0,
                        next1 => n1, next0 => n0, laser_out => x);
end architecture behavior;

```

**Quadro 5:** Código da FSM do temporizador de laser com descrição por portas lógicas.

```

library ieee;
use ieee.std_logic_1164.all;

entity reg2 is
  port(c, i1, i0 : in bit;
        q1, q0 : out bit);
end reg2;

architecture behav of reg2 is
begin
  process(c)
  begin
    if (c 'event AND c = '1') then
      q1 <= i1;
      q0 <= i0;
    end if;
  end process;
end architecture behav;

```

```

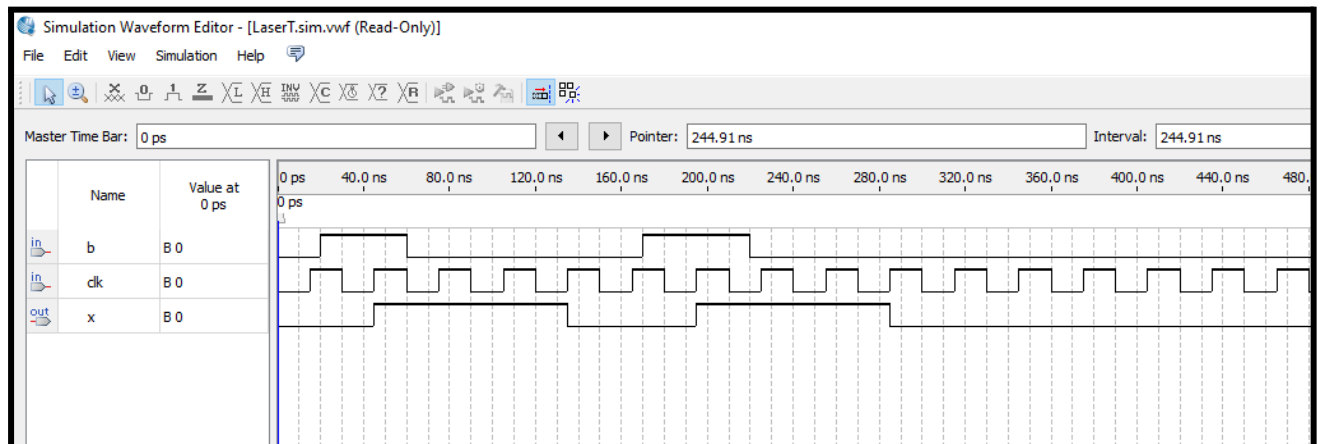
library ieee;
use ieee.std_logic_1164.all;

entity comb_circ is
  port(buttom, c, current1, current0 : in bit;
        next1, next0, laser_out : out bit);
end comb_circ;

architecture behav of comb_circ is
begin
  laser_out <= current1 OR current0;
  next1 <= (NOT(current1) AND current0)
OR (current1 AND NOT(current0));
  next0 <= (NOT(current1) AND
NOT(current0) AND buttom) OR (current1
AND NOT(current0));
end architecture behav;

```

**Quadro 6:** Componentes da descrição por portas lógicas do registrador de estado e circuito combinacional.



**Figura 15** - Diagrama de tempo do código do Quadro 6.

## **A estrutura CASE-WHEN**

A estrutura CASE-WHEN é executada de forma sequencial, podendo somente ser utilizada em regiões de código sequencial (determinada pela estrutura PROCESS). A estrutura CASE-WHEN faz com que o programa siga um dentre vários caminhos diferentes, dependendo do valor de um sinal, variável ou expressão. É uma alternativa mais elegante a uma declaração IF-THEN-ELSIF-ELSE com múltiplos ELSIF. A sintaxe básica para a estrutura CASE-WHEN é mostrada no Quadro 7.

```
case <expressão> is
    when <condição> =>
        -- código para esta condição

    when <condição> =>
        -- código para esta condição
    ...
end case;
```

**Quadro 7** – Sintaxe da estrutura CASE-WHEN da linguagem VHDL.

A <expressão> é geralmente uma variável ou um sinal. A declaração CASE pode conter várias condições para o mesmo “when”, mas apenas uma delas será selecionada.

A <condição> pode ser da seguinte forma:

- Um valor único como "11":
  - when "11" =>
- Um intervalo, como 5 a 10:
  - when 5 to 10 =>
- Pode conter vários valores como 1 ou 3 ou 5:
  - when 1 | 3 | 5 =>

- Quando nenhuma outra condição for correspondida, usamos a condição "others", a qual é equivalente ao ramo "else" na estrutura IF-THEN-ELSIF-ELSE.
  - when others =>

## Descrição comportamental em VHDL de uma FSM

O Quadro 8 mostra um modo de se modelar um bloco de controle em VHDL. O bloco de controle modelado é descrito pela FSM mostrada nas figuras 7 e 8. A entidade VHDL denominada *LaserTimer* define as entradas e saídas do bloco de controle.

A arquitetura VHDL descreve o comportamento da entidade. Ela consiste em dois processos, um que modela o registrador de estado e outro que modela a lógica combinacional. Os dois formam a arquitetura padrão do bloco de controle da Figura 11.

O primeiro processo descreve o registrador de estado do bloco de controle. Esse processo denominado *statereg* é sensível às entradas *clk* e *rst*. Se a entrada *rst* estiver habilitada, então o processo atribuirá assincronamente o estado *S\_Off* da FSM ao sinal *currentstate*. Em caso contrário, se o relógio estiver subindo, o processo atualizará o registrador de estado com o próximo estado.

Os sinais *currentstate* e *nextstate* são definidos como sendo de um tipo definido pelo usuário, de nome *statetype*. Esse tipo é definido pelo comando **type** (*tipo*) e especifica os valores possíveis que um sinal desse tipo pode assumir. Ao se especificar *statetype* para representar os estados de uma FSM, a declaração **type** listará os nomes de todos os estados do bloco de controle, especificamente *S\_Off*, *S\_On1*, *S\_On2* e *S\_On3*.

O segundo processo descreve a lógica combinacional do bloco de controle. Esse processo denominado *comblogic* é sensível às entradas da lógica combinacional da Figura 6, especificamente, às entradas externas (nesse caso, *b*) e as saídas do registrador de estado (*currentstate*). Quando qualquer um desses sinais da lista de sensibilidade sofre uma alteração, o processo coloca o valor apropriado do estado atual na saída *x*, neste caso, da FSM. O processo também determina qual deve ser o estado seguinte, baseado no estado atual e nos valores das entradas (isto é, nas condições das transições da FSM). Na próxima borda de subida do relógio, o próximo estado será carregado no registrador de estado pelo processo do registrador de estado.

Observe que a arquitetura declara os dois sinais *currentstate* e *nextstate*. Sinais são visíveis em todos os processos de uma arquitetura. O sinal *currentstate* representa o valor atual que está armazenado no registrador de estado. O sinal *nextstate* representa o valor que vem da lógica combinacional e que se dirige ao registrador de estado. Observe também que a arquitetura declara esses sinais como sendo do tipo *statetype*, o qual foi definido na arquitetura como um tipo cujo valor pode ser *S\_Off*, *S\_On1*, *S\_On2* e *S\_On3*.

A Figura 16 ilustra a forma de onda obtida com a simulação do código presente no Quadro 8. Note que a partir do instante em que a entrada *b* assume valor igual a 1, a partir da próxima borda de subida do relógio, a saída *x* permanece ativada por, exatamente, três ciclos de relógio.

```
library ieee;
use ieee.std_logic_1164.all;

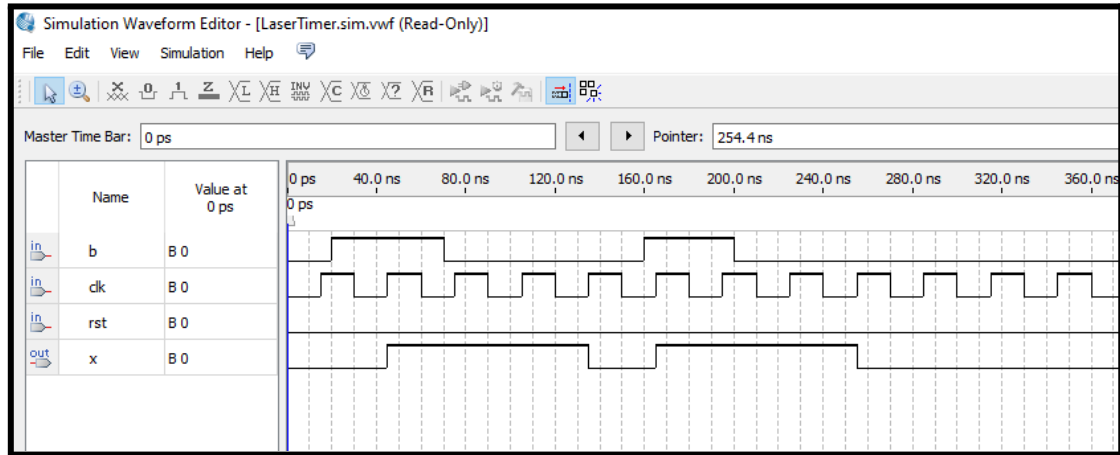
entity LaserTimer is
    port (b      : in std_logic;
          x      : out std_logic;
          clk, rst : in std_logic
        );
end LaserTimer;

architecture behavior of LaserTimer is
    type statetype is
        (S_Off, S_On1, S_On2, S_On3);
    signal currentstate, nextstate: statetype;

begin
    statereg: process(clk, rst)
    begin
        if (rst='1') then -- estado inicial
            currentstate <= S_Off;
        elsif (clk='1' and clk ' event) then
            currentstate <= nextstate;
        end if;
    end process;

    comblogic: process (currentstate, b)
    begin
        case currentstate is
            when S_Off =>
                x <='0'; -- laser desligado
                if (b='0') then
                    nextstate <= S_Off;
                else
                    nextstate <= S_On1;
                end if;
            when S_On1 =>
                x <='1'; -- laser on
                nextstate <= S_On2;
            when S_On2 =>
                x <='1'; -- laser ainda ligado
                nextstate <= S_On3;
            when S_On3 =>
                x <='1'; -- laser ainda ligado
                nextstate <= S_Off;
            end case;
        end process;
    end behavior;
```

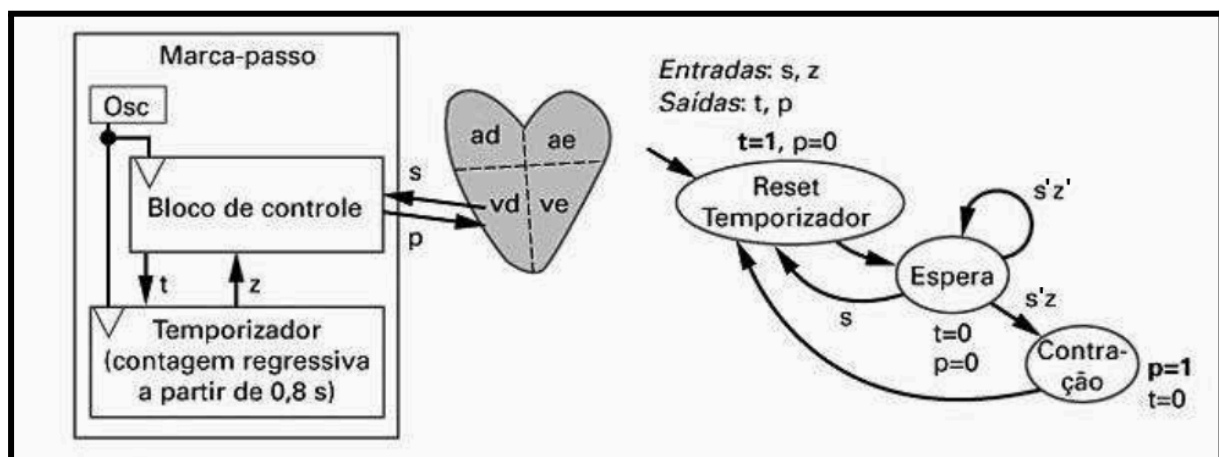
**Quadro 8:** Descrição comportamental em VHDL do bloco de controle do temporizador de laser.



**Figura 16** - Diagrama de tempo do código do Quadro 4.

## O marca-passo:

Podemos descrever o funcionamento do bloco de controle de um marca-passo simples usando a FSM da Figura 17. O lado esquerdo da figura mostra o marca-passo que consiste em um bloco de controle e um temporizador. O temporizador tem uma entrada *t*, que aplica um *reset* quando  $t=1$ . Depois do *reset*, o temporizador começa uma contagem regressiva a partir de 0,8 segundo. Se o temporizador chegar a 0, ele colocará sua saída *z* em 1. Pode acontecer do temporizador sofrer um *reset* antes de chegar a 0, caso em que o temporizador não colocará *z* em 1 e o temporizador começará uma nova contagem regressiva a partir de 0,8 segundo. O bloco de controle tem uma entrada *s*, que se torna 1 quando uma contração é detectada no ventrículo direito. O bloco de controle tem uma saída *p*, que é colocada em 1 quando o bloco de controle deve disparar uma contração.



**Figura 17** - FSM do bloco de controle de um marca-passo básico.

O lado direito da figura mostra o funcionamento do bloco de controle como uma FSM. Inicialmente, no estado *ResetTemporizador*, o bloco de controle causa um *reset* no temporizador fazendo  $t=1$ . Normalmente, o bloco de controle ficará esperando no estado *Espera* e assim permanecerá, enquanto uma contração não for detectada (*s'*) nem o temporizador tiver chegado a 0 (*z'*). Se o bloco de controle detectar uma contração natural (*s*), então ele fará novamente um *reset* no temporizador e voltará a ficar esperando. Por outro lado, se o bloco de controle ver que o temporizador chegou a 0 ( $z=1$ ), então ele seguirá para o estado *Contração*, o qual obrigará o coração a se contrair aplicando  $p=1$ . Em seguida, o bloco de controle voltará a esperar novamente. Assim, enquanto o coração estiver se contraindo naturalmente, o marca-passo não aplicará estímulo ao coração. Entretanto, se o coração não se contrair dentro de 0,8 segundo, após a última contração (natural ou forçada), o marca-passo irá forçar uma contração.

---

## **Atividades:**

- 1) Projete o bloco de controle do marca-passo da Figura 17, usando o projeto em 5 passos, descrito na Figura 10. Implemente o bloco de controle a partir de um circuito combinacional (utilizando uma descrição em VHDL por portas lógicas) + registrador de estados. Simule o circuito descrito e plote as formas de onda de suas entradas e saídas.
- 2) Implemente o bloco de controle do marca-passo da Figura 17 utilizando uma descrição comportamental em VHDL. Simule o circuito descrito e plote as formas de onda de suas entradas e saídas.
- 3) Para os circuitos implementados nos itens 1) e 2), o processo de projeto, descrição VHDL e simulação do marca passo deve ser apresentado em aula.

Considere o temporizador uma entrada externa ao circuito, que deve ter seu valor atribuído na simulação do Quartus (não precisa ser  $t=1$  a cada 0,8 s).